Indian Institute of Technology, Kanpur

Variable Selection in Multiple Non-Parametric Regression using Nadaraya Watson Estimator, with Application to Heart Disease Dataset

MTH496: Undergraduate Project - I

Piyush Verma

Sophomore, Statistics and Data Science

Under the Supervision of:

Prof. Subhra Sankar Dhar

Department of Mathematics and Statistics

April 6, 2026

Abstract

In modern data science, we frequently deal with complex, high-dimensional datasets. While including a large number of features can help uncover hidden patterns, it also slows down computation and increases the risk of overfitting. Because of this, selecting the right variables is a critical step in building reliable models.

This report explores the theory behind non-parametric regression and key model-selection tools; specifically the Akaike Information Criterion (AIC), the Bayesian Information Criterion (BIC), and the Focused Information Criterion (FIC). Building on these ideas, we propose a practical, two-stage methodology for variable screening. Our approach first uses standard parametric filtering to shrink the feature space, followed by a deeper non-parametric evaluation. By using the Nadaraya-Watson kernel estimator and Leave-One-Out Cross-Validation (LOOCV), we systematically isolate the features that hold the most genuine predictive power.

To test how well this works in practice, we applied our algorithm to the Cleveland Heart Disease dataset from the UCI Machine Learning Repository. We analyzed a cohort of 303 patient records across 13 different demographic, symptomatic, and clinical metrics. By rigorously evaluating the individual contribution of each attribute, our method successfully extracted a small, highly effective subset of features. This optimized subset maximized the model's accuracy in predicting heart disease while entirely avoiding the complications of having too many variables.

Contents

1 Introduction **1**

2 Literature Review **1**
2.1 Non-Parametric Regression . . . 1
2.2 Univariate Kernel Smoothing . . . 2
2.2.1 The Nadaraya-Watson (Local Constant) Estimator: . . . 2
2.2.2 Bandwidth Selection . . . 3
2.3 Mutli-Variate Non Parameteric Regression . . . 3
2.3.1 Curse of Dimensionality . . . 3
2.4 Information Criteria . . . 3
2.4.1 Akaike Information Criterion (AIC) . . . 4
2.4.2 Bayesian Information Criterion (BIC) . . . 4
2.4.3 Focused Information Criterion(FIC) . . . 4

3 Dataset Description **5**

4 Proposed Methodology **5**
4.1 Model Framework . . . 5
4.2 Preliminary Variable Screening . . . 5
4.3 Kernel-Based Variable Importance . . . 5
4.3.1 Focus Covariate Evaluation and Bandwidth Optimization . . . 5
4.3.2 Generalization Error Evaluation . . . 6
4.3.3 Backward Elimination . . . 6

5 Experimental Setup and Empirical Results **6**
5.1 Screening using AIC or BIC . . . 7
5.2 Optimal subset selection . . . 7
5.3 Final Predictive Performance . . . 7

6 Limitations **7**

7 Future Improvements **8**

8 Conclusion **8**

A R Code **9**

1 Introduction

In modern data science, high-dimensional datasets; such as the heart disease data analyzed in this study; often introduce the "curse of dimensionality." Including irrelevant variables unnecessarily inflates estimator variance and computational costs, making robust variable screening essential for reliable inference. To handle complex, real-world data without imposing rigid functional assumptions, researchers often turn to non-parametric regression. While highly flexible and resilient to outliers, these models are exceptionally vulnerable to the inclusion of uninformative covariates, demanding rigorous model selection techniques.

Historically, model selection has relied on global metrics like the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC), which balance overall goodness-of-fit against complexity to find a single "best" model. However, in many practical applications, the researcher's goal is much more targeted. The Focused Information Criterion (FIC) departs from this global approach by selecting the sub-model that minimizes the Mean Squared Error (MSE) for a specific parameter of interest. This localized strategy is particularly advantageous in non-parametric frameworks, where bias and variance fluctuate significantly across the covariate space. In this report, We propose a two stage methodology for variable screening.

2 Literature Review

2.1 Non-Parametric Regression

In statistical analysis, regression analysis is a statistical method for estimating the relationship between a dependent variable (often called the outcome or response variable, or a label in machine learning parlance) and one or more independent variables(often called regressors,predictors, covariates or features). Basically, it involves modelling how the expected value of a response y changes in response to changes in one or more explanatory variable(s) $\tilde{\mathbf{X}}$. Mathematically,

$$\mathbb{E}(y \mid x) = f(x)$$

Traditional Regression methods involved assuming a finite set of unknown parameters and a functional form and then mapping the input covariates to response variable. This is what we call **Parametric Regression**. One of the most famous and starting example of Parametric regression is **Linear Regression** which has the following mathematical equation:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon} \tag{1}$$

where:

- $\mathbf{y}$ is an $n \times 1$ response vector (the dependent variable),
- $\mathbf{X}$ is an $n \times p$ design matrix (the features or explanatory variables),
- $\boldsymbol{\beta}$ is a $p \times 1$ vector of unknown parameters (the regression coefficients),
- $\boldsymbol{\epsilon}$ is an $n \times 1$ vector of unobserved random errors.

The main issue with parametric regression is that we have to preassume a functional form, which is not always feasible. **Nonparametric regression** solves this problem and offers a flexible alternative to parametric methods for regression.The basic idea of nonparametric inference is to use data to infer an unknown quantity while making as few assumptions as possible. Usually, this means using statistical models that are infinite-dimensional. Mathematically, consider an independent and identically distributed sample $(X_1, Y_1), \ldots, (X_n, Y_n)$. Non-parametric regression is then defined by:

$$Y_i = m(X_i) + \epsilon_i, \quad \text{for } i = 1, 2, \ldots, n \tag{2}$$

where:

- Y_i is the response (or dependent) variable for the i-th observation,
- X_i is the predictor (or independent) variable for the i-th observation,
- $m(\cdot)$ is an unknown, smooth regression function representing the conditional expectation, such that $m(x) = \mathbb{E}(Y_i \mid X_i = x)$,

- ϵ_i is the unobserved random error term.

To ensure the model is well-defined, we typically impose the following standard assumptions on the error term ϵ_i:

- $\mathbb{E}(\epsilon_i \mid X_i) = 0$ (the errors have a conditional mean of strictly zero),
- $\mathrm{Var}(\epsilon_i \mid X_i) = \sigma^2(X_i)$ (the errors have a finite variance, which may be constant or depend on the covariate X_i).

2.2 Univariate Kernel Smoothing

As discussed, **non-parametric regression (NP regression)** allows the data itself to dictate the general shape of the relationship without imposing strict functional assumptions. Our goal is to predict the response value for any given covariate x, utilizing a finite sample: $(x_1, y_1), \ldots, (x_n, y_n)$.

NP regression seeks an estimator for the unknown regression function $m(\cdot)$. A natural way to estimate $m(x)$ at a specific point x is through local averaging—taking a weighted average of the observed y_i values corresponding to x_i values that fall within a local neighborhood of x. This is given by:

$$\hat{m}(x) = \frac{\sum_{i=1}^{n} y_i w_i(x)}{\sum_{i=1}^{n} w_i(x)}$$

where $w_i(x)$ represents the weight assigned to the i-th observation when predicting at point x.

The term **kernel** (denoted by K) is generally used to refer to the weighting function. It refers to any smooth function such that $K(x) \geq 0$, and

$$\int K(x)\,dx = 1, \quad \int xK(x)\,dx = 0,$$
$$\text{and} \quad \sigma_K^2 \equiv \int x^2 K(x)\,dx > 0.$$

Some commonly used kernels are the following:

- **The boxcar kernel:** $K(x) = \frac{1}{2} I(x)$
- **The Gaussian kernel:** $K(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$

Another hyperparameter h, known as the *bandwidth parameter*, is used to scale the kernel. The bandwidth parameter controls the smoothness of the resulting regression estimate by determining the effective width of the kernel applied to each data point. In simple terms, it dictates the size of the local neighborhood and controls the smoothness of the estimate.

Kernel Density Estimation(KDE) is the application of kernel smoothing for probability density estimation, i.e., a non-parametric method to estimate the probability density function of a random variable based on kernels as weights. let $\tilde{X}$=$(x_1, x_2, \ldots)$independent and identically distributed samples drawn from some univariate distribution with an unknown density f at any given point x. Then, *Kernel density estimator* is given by:

$$\hat{f}_h(x) = \frac{1}{n}\sum_{i=1}^{n} K_h(x - x_i) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right),$$

where K is the *kernel* and h>0 is the *bandwidth parameter*
Some well known estimators(often called as *smoothers*):

2.2.1 The Nadaraya-Watson (Local Constant) Estimator:

Nadaraya and Watson proposed to estimate m as a locally weighted average, using a kernel as a weighting function. The nardaraya watson estimator is given by:

$$\widehat{m}_h(x) = \frac{\sum_{i=1}^{n} K_h(x - x_i) y_i}{\sum_{i=1}^{n} K_h(x - x_i)}$$

where $K_h(t) = \frac{1}{h} K\left(\frac{t}{h}\right)$ is a kernel with a bandwidth h such that $K(\cdot)$ is of order at least 1, that is $\int_{-\infty}^{\infty} uK(u)\,du = 0$.

Asymptotic properties of Nadaraya Watson estimator are of particular interest. some of the well known asymptotic results of NW estimator

are:

$$\text{Bias}(\hat{m}_h(x)) \approx h^2\mu_2(K) \times \left(\frac{1}{2}m''(x) + \frac{m'(x)f'(x)}{f(x)}\right)$$

$$\text{Var}(\hat{m}_h(x)) \approx \frac{1}{nh}\frac{\sigma^2(x)}{f(x)}R(K)$$

NW estimator suffers from a significant structural limitation known as **design bias**. The term $\frac{m\prime(x)f\prime(x)}{f(x)}$ in asymptotic bias is called the *design bias* because it demonstrates that bias is directly influenced by distribution(design) of the data.

2.2.2 Bandwidth Selection

The **Bandwidth parameter h** controls the smoothness of the local constant estimator.

- **Small h**: Captures fine local details but increases variance, therefore has potential risk of overfitting.
- **Large h**: Smooths out noise but may miss important local structure, and therefore has potential risk of underfitting.

The parameter h is selected using **Cross Validation**. The **Leave-One-Out Cross Validation(LOOCV)** is given by:

$$\text{CV}(h) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{m}_{-i,h}(x_i))^2$$

where $\hat{m}_{-i,h}(x_i)$ is the estimate at x_i obtained by leaving out the ith observation.

2.3 Mutli-Variate Non Parameteric Regression

Most of the time, regressor usually exists in $\mathbb{R}^d$ rather than simply existing in $\mathbb{R}$ space. Consider n sample points in $\mathbb{R}^d$, (X_i, Y_i) , where $X_i \in \mathbb{R}^d$, then consider the multivariate non parameteric regression model as:

$$Y_i = m(X_{i1}, X_{i2}, \ldots, X_{id}) + \epsilon_i$$

where $m(\cdot) : \mathbb{R}^d \to \mathbb{R}$ is an unknown, smooth function and ϵ_i are iid, conditioned on X.

Multi-Variate Kernel Smoothing : Similar to univariate kernel smoothing, multivariate kernel smoothing involves using a multivariate kernel as weighing function. Usually the multivariate kernel is simply taken as a product of univariate kernels applied on each of the d features, with flexibility of choosing a different bandwidth parameter h_j for each feature.The expression is:

$$K_H(u) = \prod_{j=1}^{d}\frac{1}{h_j}\cdot k(\frac{u_j}{h_j})$$

2.3.1 Curse of Dimensionality

The curse of dimensionality in nonparametric regression describes decrease in fastest achievable rates of convergence of regression function estimators toward their target curve as the dimension of the regressor vector increases. Because data becomes extremely sparse in high-dimensional spaces, local smoothing methods require an exponential increase in sample size to maintain accuracy. Consider the data have dimension d, and sample size n. The MSE of any non-parametric estimator of a smooth(twice differentiable) curve typically has the following form:

$$\text{MSE} \approx \frac{c}{n^{\frac{4}{4+d}}}$$

for some c > 0. So if we want MSE to be equal to some δ then we find that:

$$cn = (\frac{c}{\delta})^{\frac{d}{4}}$$

which grows with exponential with dimension d.

This implies that dimension reduction is crucially required while working with a finite sample of high dimensional data.

2.4 Information Criteria

An **Information Criterion (IC)** is a mathematical ruler used to evaluate and compare the quality of different statistical models for a given set of data. Any statistical model, if provided with more features, will always improve its fit on the training set (in other words, it will decrease

bias), but it increases the risk of overfitting. The model may start capturing noise as the true signal. Information criteria provide a formalized way to navigate this bias-variance trade-off.

The theoretical backbone of most information criteria is the Kullback-Leibler (K-L) Divergence between the true, unknown data-generating distribution, $f(y)$, and an approximate model, $g(y|\theta)$. The IC evaluates the model based on two opposing forces: *goodness-of-fit* and a penalty for the *complexity* of the model.

The generalized mathematical expression for an information criterion is given by:

$$\text{IC} = -2\ln(\hat{L}) + \text{Penalty}(k, n)$$

where:

- $\hat{L}$ is the maximized value of the likelihood function for the model.
- k is the number of estimated parameters in the model.
- n is the sample size.

The term $-2\ln(\hat{L})$ represents the "badness of fit" (deviance). A better-fitting model will have a higher likelihood and a lower deviance. The term $\text{Penalty}(k, n)$ penalizes the model for its complexity.

The main goal is to select the model with the lowest IC score. Note that IC values are strictly comparative; these values do not carry any inherent meaning of their own.

2.4.1 Akaike Information Criterion (AIC)

AIC estimates the relative amount of information lost by a given model when used to approximate the true, unknown data-generating process.

$$\text{AIC} = -2ln(\hat{L}) + 2k$$

AIC is designed for prediction. It tries to select the model that will most accurately predict future data. It is asymptotically equivalent to leave-one-out cross-validation.

2.4.2 Bayesian Information Criterion (BIC)

BIC is derived from bayesian probability.

$$\text{BIC} = -2ln(\hat{L}) + kln(n)$$

BIC is designed for model identification. It assumes that the true data-generating model is actually in your set of candidate models. As the sample size n grows, BIC guarantees that it will select the true model with a probability approaching 1.Because $\ln(n) > 2$ for any $n \geq 8$, BIC penalizes complex models much more harshly than AIC.

2.4.3 Focused Information Criterion(FIC)

Traditional model selection criteria like AIC and BIC are designed to find the best overall fit for an entire dataset. However, in practice, we often build models to answer a specific question or estimate a single target, known as the focus parameter (denoted as μ). Introduced by Claeskens and Hjort (2003), the **Focused Information Criterion (FIC)** addresses this by shifting the goalpost. Instead of asking which model is the most universally "true," FIC identifies the sub-model that provides the most accurate estimate for our specific focus.

At its core, FIC navigates the classic bias-variance tradeoff by estimating the Mean Squared Error (MSE) for the focus parameter across all candidate models. For a given candidate model S, the MSE of its focus parameter estimator, $\hat{\mu}_S$, is defined as:

$$MSE(\hat{\mu}_S) = \text{Bias}(\hat{\mu}_S)^2 + \text{Var}(\hat{\mu}_S)$$

In this formula, the squared bias represents the error introduced by simplifying the model (underfitting), while the variance measures the uncertainty or fluctuation in the estimate caused by including too many variables (overfitting).

FIC uses the sample data to provide an empirical estimate of this MSE. By calculating the FIC score for every candidate, we effectively rank them by how well they estimate our specific parameter of interest. Ultimately, the model with the lowest FIC score is selected.

3 Dataset Description

To evaluate the performance of our model, we utilized the **Cleveland Heart Disease dataset,** sourced from the UCI Machine Learning Repository. The analysis was conducted on a cohort of 303 patient records. The original dataset has the target variable is integer valued from 0 (no presence) to 4, to represent increasing levels of disease severity. For the purpose of binary classification, the target variable was structured to distinguish between 164 individuals exhibiting no evidence of heart disease ($Y = 0$) and 139 individuals diagnosed with the condition ($Y = 1$).

The predictive framework relies on a 13-dimensional feature space that captures a comprehensive mix of demographic, symptomatic, and clinical diagnostic metrics. Specifically, the model inputs consist of the patient's age, sex, chest pain classification(*cp*), resting blood pressure(*trestbps*), and serum cholesterol(*chol*). It further incorporates specialized test results, including fasting blood sugar levels(*fbs*), resting electrocardiogram outputs(*restecg*), maximum heart rate achieved(*thalach*), the presence of exercise-induced angina(*exang*), ST depression relative to rest (*oldpeak*), the slope of the peak exercise ST segment(*slope*), the number of major vessels identified via fluoroscopy(*ca*), and thalassemia type(*thal*). Together, these attributes establish the baseline covariate vector used to train and test the algorithm.

4 Proposed Methodology

4.1 Model Framework

Let $\{(X_i, Y_i)\}_{i=1}^n$ represent an independent and identically distributed (i.i.d.) sample. Although the final target is binary ($Y \in \{0, 1\}$), we formulate this as a regression problem. Consequently, the unknown function $m(x) = E[Y|X = x]$ reflects the conditional probability of the target, which can subsequently be thresholded to produce discrete classifications. The underlying model is defined as:

$$Y_i = m(X_{i1}, X_{i2}, \ldots, X_{ip}) + \epsilon_i, \quad i = 1, \ldots, n$$

where $m(\cdot)$ is assumed to be a sufficiently smooth function, and the error term satisfies $E[\epsilon_i|X_i] = 0$.

Letting $A = \{1, 2, \ldots, p\}$ denote the complete index set of all available covariates, we define a restricted subvector for any subset $S \subseteq A$ as $X_{i,S} = \{X_{i,j} : j \in S\}$. Our primary objective is to discover the optimal, most parsimonious feature subset $S^* \subset A$ that maximizes predictive accuracy while minimizing dimensionality.

4.2 Preliminary Variable Screening

Prior to executing the non-parametric feature selection, we conduct a preliminary screening using information criteria (such as AIC and BIC). This initial phase effectively filters out covariates that lack statistical significance or exhibit high multicollinearity, thereby strictly narrowing down the full index set A into a mathematically tractable subset, denoted as S_0. All subsequent kernel-based evaluations are executed exclusively within this diminished feature space.

4.3 Kernel-Based Variable Importance

Following the initial dimensionality reduction, we introduce a non-parametric screening framework to evaluate the isolated predictive strength of each remaining feature without imposing rigid parametric forms.

4.3.1 Focus Covariate Evaluation and Bandwidth Optimization

Instead of modeling all variables simultaneously, we isolate each index $k \in S_0$ as a *focus covariate*, X_k, to estimate its independent, marginal effect on the response:

$$\hat{Y}_k = \hat{E}[Y|X_k] = \hat{m}_k(X_k)$$

To construct the estimator $\hat{m}_k(\cdot)$, we employ the Nadaraya-Watson (Local Constant) kernel

smoothing method. This allows the model to fluidly adapt to the data's intrinsic structure. Crucially, the accuracy of this estimator depends heavily on the bandwidth parameter h_k. To rigorously manage the bias-variance tradeoff, we avoid using a static global bandwidth. Instead, we assign an optimal, feature-specific bandwidth h_k^* for each focus variable via Data-Driven Least-Squares Cross-Validation (LSCV):

$$h_k^* = \arg\min_h \text{LSCV}(h)$$

4.3.2 Generalization Error Evaluation

With the optimal bandwidth h_k^* determined, we quantify the predictive vulnerability of each covariate by calculating an unbiased estimate of its out-of-sample risk. This is achieved via the Leave-One-Out Cross-Validation (LOOCV) Mean Squared Error:

$$\text{MSE}_{CV}(k) = \frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \hat{m}_{-i}(X_{i,k}; h_k^*)\right)^2$$

Here, $\hat{m}_{-i}(X_{i,k}; h_k^*)$ represents the Nadaraya-Watson estimate for the i-th observation, computed by deliberately excluding that specific data point from the training set. The resulting MSE reflects the average squared prediction error across all left-out folds. A lower MSE_{CV} indicates superior isolated predictive power for that specific covariate.

4.3.3 Backward Elimination

The covariates are subsequently sorted based on their LOOCV MSE in ascending order. Adopting a backward elimination strategy, the variable responsible for the largest error—signifying the weakest marginal predictive capability—is deemed to contain the least structural information and is permanently discarded. The surviving variables form the optimized, parsimonious subset S^*, which is ultimately utilized to train the final predictive classifier.

Methodology Summary

To synthesize the proposed framework, the feature selection process is executed through the following sequential steps:

1. **Problem Formulation:** The binary classification task is framed as a non-parametric regression problem to estimate the conditional probability $m(x) = E[Y|X = x]$.

2. **Parametric Pre-screening:** Information criteria (e.g., AIC, BIC) are applied to filter out irrelevant or highly multicollinear features, reducing the initial feature space A to a mathematically tractable subset S_0.

3. **Bandwidth Optimization & Marginal Estimation:** For each covariate in S_0, an optimal, feature-specific bandwidth h_k^* is determined using Least-Squares Cross-Validation (LSCV). The variable's independent predictive effect is then modeled using a Nadaraya-Watson kernel estimator.

4. **Cross-Validated Risk Assessment:** The Leave-One-Out Cross-Validation (LOOCV) Mean Squared Error is calculated for each independently modeled feature to objectively quantify its out-of-sample predictive strength.

5. **Iterative Feature Pruning:** The covariates are ranked by their LOOCV MSE. Using backward elimination, the variable with the highest error is iteratively discarded until the most parsimonious and highly predictive feature subset, S^*, is achieved.

5 Experimental Setup and Empirical Results

The entire algorithm involved many computational steps, which were optimized for efficiency using R packages, `foreach` and `doParallel`. Before any feature selection methodology was applied, the dataset was partitioned into a training subset(80%) and a testing subset(20%) for evaluation purposes.

5.1 Screening using AIC or BIC

Initially, we tested both AIC and BIC for our preliminary feature selection. The results using AIC,and the BIC were very similar. Because BIC has a stricter penalty, it naturally resulted in a smaller total subset of features, stopping at k=6 instead of k=8. However, the key takeaway is that both methods achieved the exact same highest accuracy (86.67%) at an initial subset size of k=4. At this specific step, both AIC and BIC kept the exact same features and dropped the same variable (exang). Since the choice between AIC and BIC does not change our final optimal model at all, we decided to just proceed with AIC for the rest of the methodology.

Table 1: **Iterative Evaluation of Subset Size (k) using BIC**

Initial k	**Dropped Var.**	**Size**	**Accuracy**
3	exang	2	78.33%
4	**exang**	**3**	**86.67%**
5	sex	4	85.00%
6	**sex**	**5**	**86.67%**

5.2 Optimal subset selection

To determine the optimal number of features in the final subset that achieves the highest accuracy, an iterative study was conducted for $k \in \{3, 4, ..., 8\}$, where 8 is the number of features attained after preliminary screening using AIC. The Nadaraya Watson estimation and LOOCV feature-elimination algorithm was carried out for each value of k.

Table 2: **Iterative Evaluation of Subset Size (k) using AIC**

Initial k	**Dropped Var.**	**Size**	**Accuracy**
3	exang	2	78.33%
4	**exang**	**3**	**86.67%**
5	sex	4	85.00%
6	**sex**	**5**	**86.67%**
7	sex	6	85.00%
8	**trestbps**	**7**	**86.67%**

To reduce the curse of dimensionality in the final model, the subset size $k = 4$ was chosen since it achieves the maximum accuracy of 86.67% without increasing the number of covariates. The final size of the subset is 3 after dropping the *exang* variable.

5.3 Final Predictive Performance

To finally evaluate the effectiveness of our variable selection methodology, logistic regression model was finally trained using the training dataset containing only the final three selected features(*thal, ca, and oldpeak*). The model achieved a remarkable accuracy of **86.67%**. To further illustrate the validity of the methodology, a negative control model was trained on the rest of the 10 discarded features. It achieved an accuracy of only 71.67% which is far less than what our parsimonious model achieved.

6 Limitations

- **The Univariate isolation flow:** In Stage 2, the Nadaraya-Watson (NW) kernel regression evaluates each feature in complete isolation. This assumes that a variable's predictive power is entirely independent of others. It completely misses feature interactions (e.g., if age and cholesterol are only dangerous when combined). A feature might have a terrible univariate MSE but be highly predictive in a multivariate space.
- **Small dataset and high variance:** Due to the limited size of the UCI dataset,

our final holdout test set contained approximately 60 observations. While our nonparametric model demonstrated strong performance on this holdout, a sample size of this magnitude introduces variance into the final accuracy metric. A different random partition of the data could yield fluctuating performance scores.

7 Future Improvements

- **Implement Multivariate Additive Auditing**: Instead of testing features one by one, future iterations could use Generalized Additive Models (GAMs) or multivariate kernel regression. This would allow the algorithm to penalize variables while accounting for their interactions with the rest of the subset, maintaining the structural integrity of the feature space.
- **Imputation and Robust validation** Instead of dropping rows with missing values, implement a pre-processing step using K-Nearest Neighbors (KNN) imputation to preserve the statistical power of the small dataset. Furthermore, the 80/20 train/test split can be replaced with k-fold cross-validation (e.g., 5-fold or 10-fold) for a better evaluation.
- **Non-Linear Final Classification:** To truly capitalize on the nonparametric screening, the final evaluation model should be capable of mapping non-linear decision boundaries. Replacing the final glm with a GAM, or a Random Forest would ensure the non-linear signals isolated in Stage 2 are actually utilized in the final prediction.

8 Conclusion

Because nonparametric (NP) regression is computationally expensive, it is essential to minimize the feature space while still maintaining a low model bias. This project primarily focused on addressing this critical challenge in feature selection.

We tested our methodology on a heart disease dataset and successfully extracted a subset of just three features. Notably, this refined model outperformed a broader baseline of ten features in predicting the likelihood of the disease.

This algorithm can be readily applied to similar domains that rely on high-dimensional data, such as healthcare and credit risk assessment. By isolating the most predictive variables, it significantly reduces the computational cost and time required to train advanced statistical and machine learning models.

References

[1] Wasserman, L. (2006). *All of Nonparametric Statistics*. Springer Science & Business Media.

[2] Fan, J., & Gijbels, I. (1996). *Local Polynomial Modelling and its Applications*. CRC Press.

[3] Stone, C. J. (1982). Optimal global rates of convergence for nonparametric regression. *The Annals of Statistics*, 10(4), 1040-1053.

[4] Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716-723.

[5] Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461-464.

[6] Aho, K., Derryberry, D., & Peterson, T. (2014). Model selection for ecologists: the worldviews of AIC and BIC. *Methods in Ecology and Evolution*, 5(1), 14-26.

[7] Claeskens, G., & Hjort, N. L. (2003). The focused information criterion. *JASA*, 98(464), 900-916.

[8] Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). Heart Disease [Dataset]. UCI Machine Learning Repository. `https://doi.org/10.24432/C52P4X`.

A R Code

```r
library(MASS)
library(np)
library(foreach)
library(doParallel)

# Silence the continuous bandwidth optimization printouts
options(np.messages = FALSE)

# ==========================================
# 1. Data Preparation (UCI Heart Disease Dataset)
# ==========================================
cat("Loading UCI Heart Disease (Cleveland) dataset...\n")
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"
col_names <- c("age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
               "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target")

# Load data and handle missing values denoted by "?"
data <- read.csv(url, header = FALSE, col.names = col_names, na.strings = "?")
data <- na.omit(data) # Clean dataset to ensure CV runs perfectly

# Encode target variable (0 = No disease, 1 = Disease present)
Y <- ifelse(data$target > 0, 1, 0)
X <- data[, 1:13]

# ==========================================
# 2. Train / Test Partitioning (80/20 Split)
# ==========================================
set.seed(496) # Fixed seed for strict reproducibility
train_idx <- sample(1:nrow(X), size = 0.8 * nrow(X))

X_train <- X[train_idx, ]
Y_train <- Y[train_idx]
X_test  <- X[-train_idx, ]
Y_test  <- Y[-train_idx]

# ==========================================
# 3. Stage 1: Parametric Screening (AIC)
# ==========================================
null_model <- glm(Y_train ~ 1, data = X_train, family = binomial)
full_model <- glm(Y_train ~ ., data = X_train, family = binomial)

# Forward-stepwise selection using AIC penalty (2k)
aic_model <- step(null_model,
                  scope = list(lower = null_model, upper = full_model),
                  direction = "forward",
                  trace = 0)

selected_features <- names(coef(aic_model))[-1]

# Restricting to top 4 features since they achieve the best accuracy
selected_features <- selected_features[1:4]

cat("\n=====================================================\n")
cat("STAGE 1: AIC PARAMETRIC SCREENING\n")
```

```r
cat("Optimal subset isolated after Stage 1:\n")
print(selected_features)
cat("=====================================================\n\n")

X_sub_train <- X_train[, selected_features, drop=FALSE]

# ==========================================
# 4. Stage 2: Nonparametric LOOCV using Nadaraya-Watson
# ==========================================
cores <- min(parallel::detectCores() - 1, length(selected_features))
cl <- makeCluster(cores)
registerDoParallel(cl)

cat("Evaluating features using Nadaraya-Watson (Local Constant) Cross-Validation
    ...\n")

# Isolate predictive power of each focus variable via Kernel Smoothing
results <- foreach(focus_var = selected_features,
                   .combine = rbind, .packages = "np") %dopar% {

                     x_focus <- X_sub_train[[focus_var]]

                     # Regress Y on the focus variable using Nadaraya-Watson
                     bw_out_nw <- npregbw(xdat = x_focus, ydat = Y_train,
                                          regtype = "lc", bwmethod = "cv.ls")

                     # Extract Out-of-Sample Mean Squared Error
                     mse_nw <- bw_out_nw$fval

                     data.frame(Variable = focus_var, MSE_NW = mse_nw)
                   }

stopCluster(cl)

# ==========================================
# 5. Variable Ranking & Elimination
# ==========================================
cat("\n--- Predictive Errors (Nadaraya-Watson LOOCV MSE) ---\n")
print(results)

# Sort by Mean Squared Error (Descending)
ranked_nw <- results[order(-results$MSE_NW), ]

# The variable with the highest error contains the least structural info
var_to_drop_nw <- ranked_nw$Variable[1]

cat("\n--- Nonparametric Feature Elimination ---\n")
cat("Highest Error (Least Info):", var_to_drop_nw, "-> Dropped.\n")

# ==========================================
# 6. Final Validation & Negative Control
# ==========================================
final_features <- setdiff(selected_features, var_to_drop_nw)
dropped_features <- setdiff(colnames(X), final_features)

# Train Parsimonious Model (Optimal features)
final_model <- glm(Y_train ~ ., data = X_train[, final_features, drop=FALSE],
    family = binomial)
```

```r
# Train Negative Control Model (Discarded features)
control_model <- glm(Y_train ~ ., data = X_train[, dropped_features, drop=FALSE], family = binomial)

# Predict classifications on unseen testing data
pred_final_prob <- predict(final_model, newdata = X_test[, final_features, drop=FALSE], type = "response")
pred_final_class <- ifelse(pred_final_prob > 0.5, 1, 0)

pred_control_prob <- predict(control_model, newdata = X_test[, dropped_features, drop=FALSE], type = "response")
pred_control_class <- ifelse(pred_control_prob > 0.5, 1, 0)

# Evaluate global test accuracy
acc_final <- mean(pred_final_class == Y_test)
acc_control <- mean(pred_control_class == Y_test)

cat("\n=====================================================\n")
cat("METHODOLOGY PERFORMANCE\n")
cat("=====================================================\n")
cat(sprintf("Parsimonious Model (%d Variables): %.2f%%\n", length(final_features), acc_final * 100))
cat(sprintf("Negative Control (%d Variables): %.2f%%\n", length(dropped_features), acc_control * 100))
```